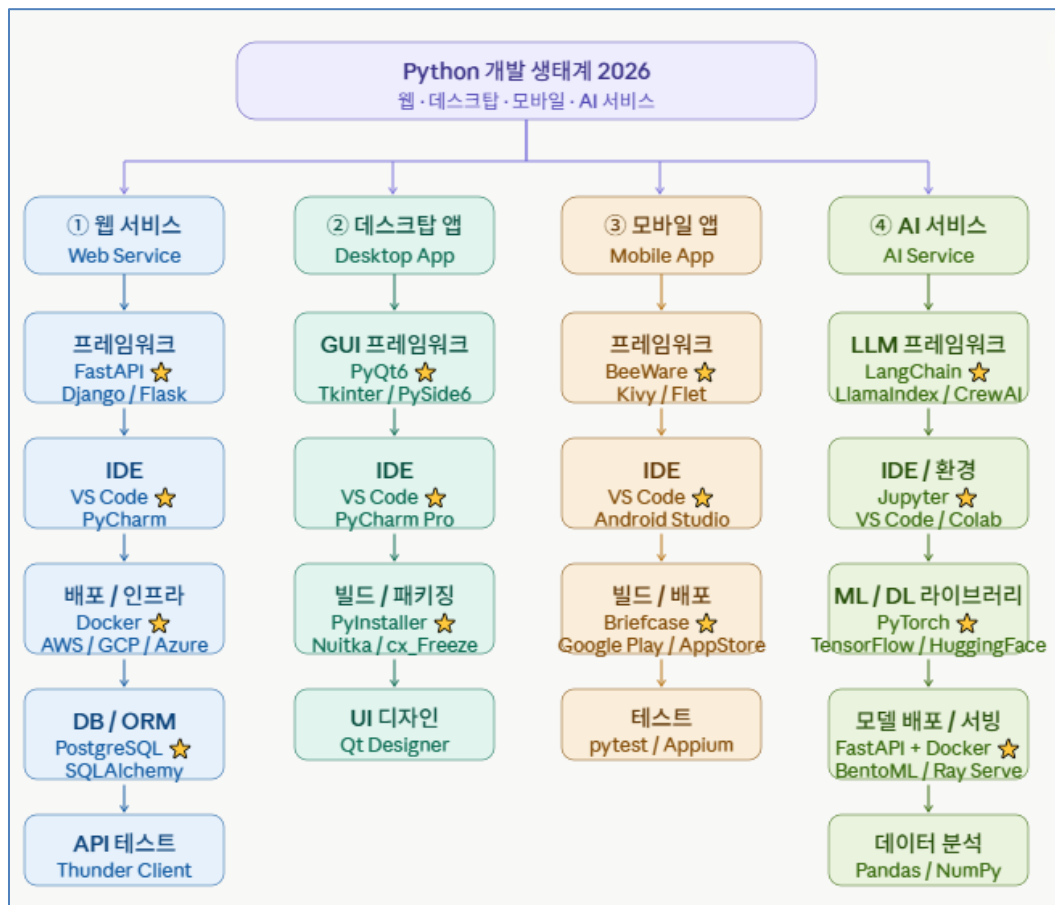




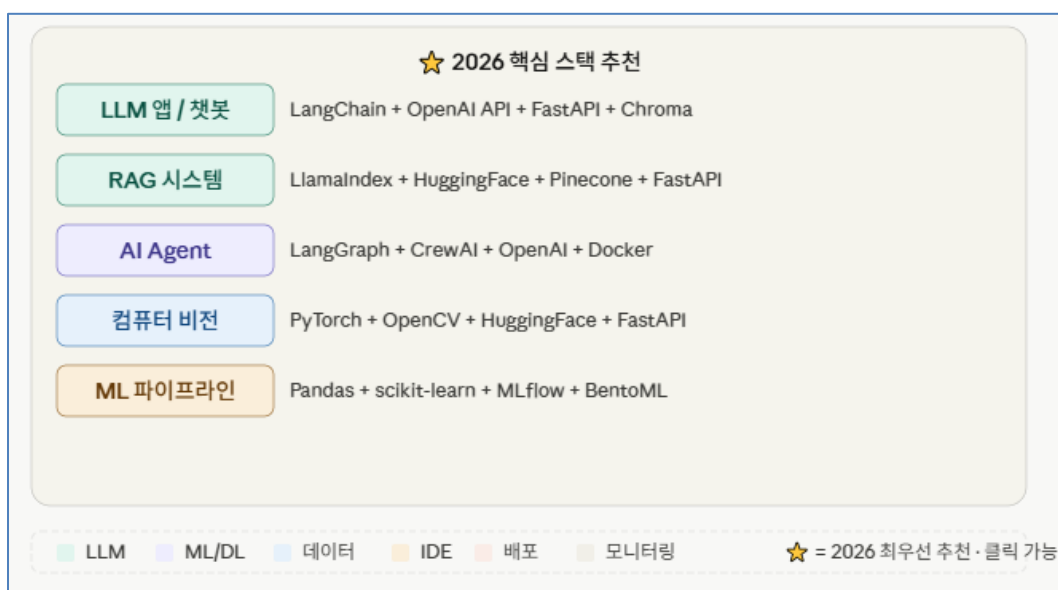
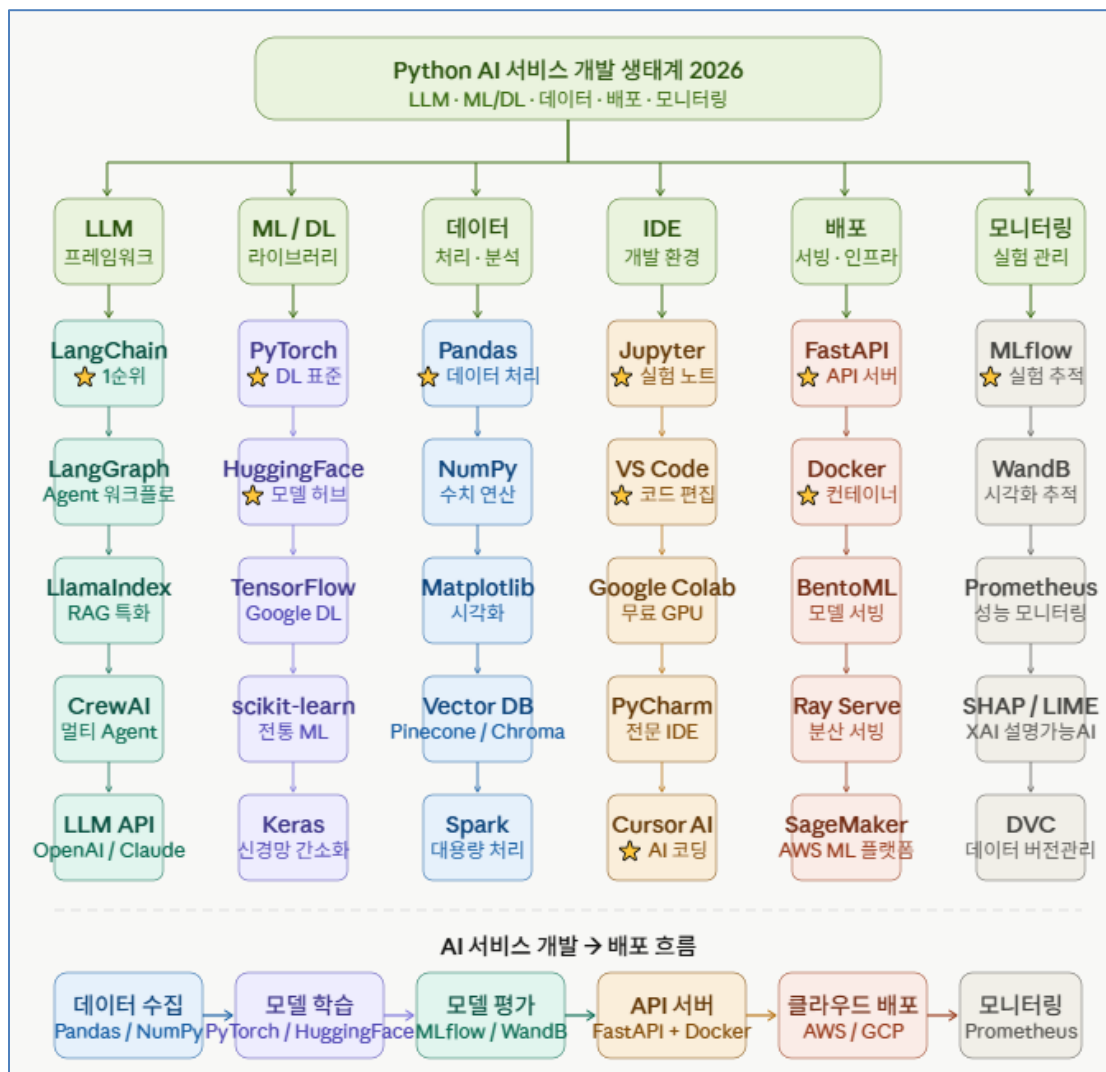


① **웹 서비스** — 가장 진입장벽이 낮아요. FastAPI 가 2026 기준 압도적 트렌드이고, Django 는 대형 프로젝트에 여전히 강력해요.

② **데스크탑 앱** — PyQt6 가 가장 성숙하고 상업용으로 검증됐어요. UI 를 Qt Designer 로 시각적으로 만들 수 있어서 생산성이 높아요.

③ **모바일 앱** — Python 으로 모바일 개발은 아직 제한적이에요. BeeWare/Flet 이 가능하지만, 실무에서는 Flutter(Dart)나 React Native 가 더 현실적인 선택이에요.

④ **AI 서비스** — Python 이 독보적으로 강한 분야예요. LangChain + PyTorch + HuggingFace 조합이 2026 표준 스택이에요.



① **LLM 프레임워크** — LangChain 이 GPT-4, Claude 같은 대형 언어 모델 기반 앱 개발의 표준으로 자리잡았어요. 여기에 Agent 워크플로 전용인 LangGraph, 문서 검색(RAG) 특화인 LlamaIndex, 멀티 에이전트 조율에 강한 CrewAI 가 2026 핵심 4 인방이에요. [Softjournal](#)

② **ML/DL 라이브러리** — PyTorch 와 TensorFlow 가 딥러닝 모델 개발·배포의 양대 산맥이고, HuggingFace Transformers 는 수천 개의 사전학습 모델에 접근할 수 있는 필수 허브가 됐어요. [ClarionTech](#)

③ **데이터 처리** — Pandas + NumPy 가 기본이고, RAG 시스템에는 **Vector DB**(Pinecone, Chroma)가 필수로 추가됐어요.

④ **IDE / 개발환경** — Jupyter 는 실험·분석용, VS Code 는 코드 작성용으로 분리해서 쓰는 게 표준이에요. 2026 년엔 Cursor AI 같은 AI 코딩 보조 IDE 가 생산성 표준으로 급부상하고 있어요. [Dasroot](#)

⑤ **배포** — FastAPI + Docker 조합이 AI 모델 API 서버의 가장 현실적인 선택이고, LangChain/LangGraph 와 결합한 에이전트 기반 배포가 엔터프라이즈 표준으로 자리잡고 있어요. [Dasroot](#)

⑥ **모니터링** — MLflow 로 실험을 추적하고, SHAP/LIME 같은 설명 가능한 AI(XAI) 도구가 의료·금융·법률 분야에서 필수로 요구되고 있어요.

Flask AI Portfolio

완전 초보자를 위한 Step-by-Step 개발 매뉴얼

Python 3.13 | Flask | SQLite3 | Jinja2 | JavaScript

2025 년 기준 최신 소스코드 반영

목 차

- 1 장 프로젝트 개요 및 구성
- 2 장 개발 환경 설치
- 3 장 프로젝트 폴더 구조
- 4 장 설정 파일 — config.py
- 5 장 데이터베이스 계층 — database.py
- 6 장 라우트 등록 — routes.py
- 7 장 서버 진입점 — app.py
- 8 장 템플릿 — templates/
- 9 장 정적 파일 — static/
- 10 장 날씨 위젯 — main.js
- 11 장 관리자 페이지
- 12 장 에러 페이지 (404 / 500)
- 13 장 서버 실행 및 테스트
- 14 장 환경 변수 및 보안
- 부록 SQLite 직접 조회 방법

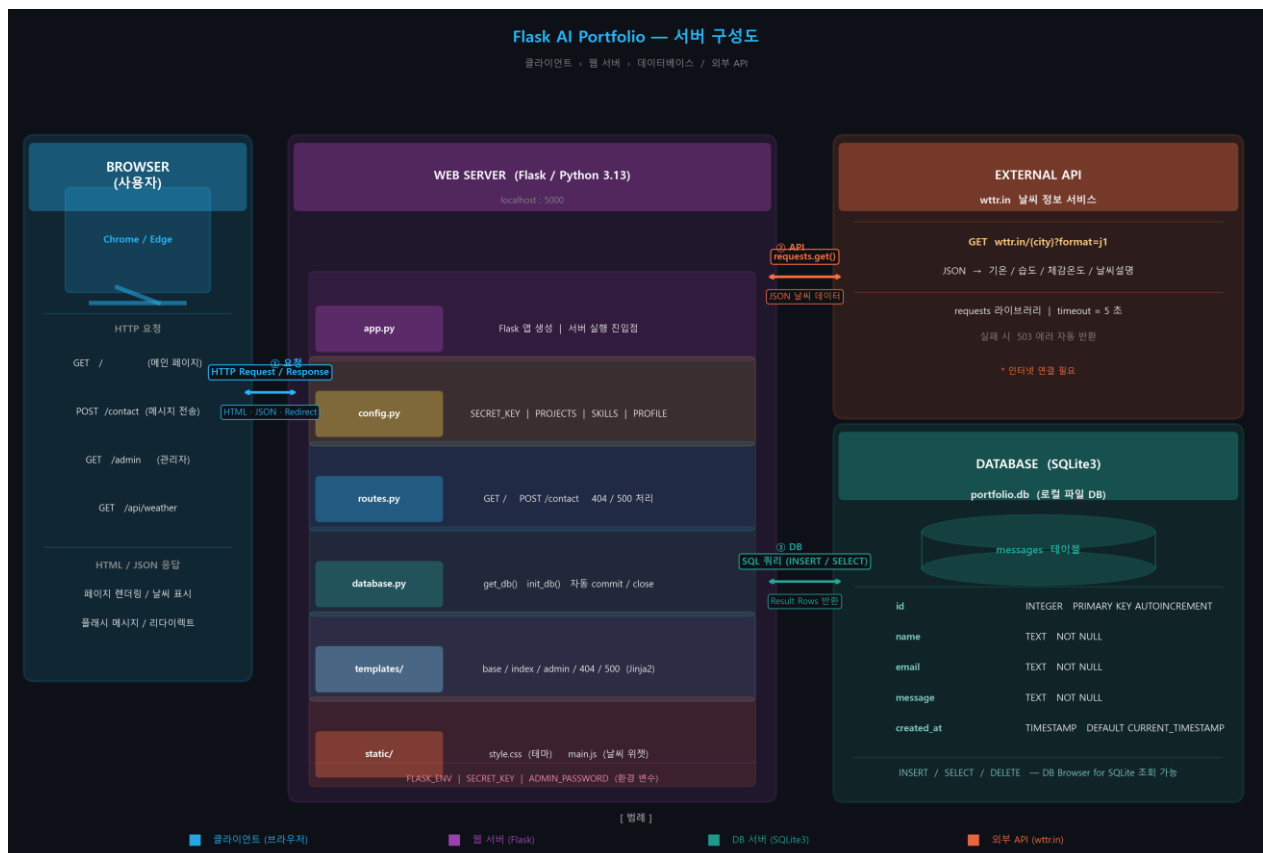
1 장 프로젝트 개요 및 구성

이 프로젝트는 Python Flask 웹 프레임워크를 사용해 만든 AI 개발자 포트폴리오 웹사이트입니다. 외부 날씨 API 연동, SQLite 데이터베이스, Jinja2 템플릿 엔진을 모두 활용합니다.

1-1 주요 기능

- 메인 페이지 — 프로필, 기술 스택, 프로젝트 소개
- 날씨 위젯 — 실시간 서울 날씨 표시 (wttr.in API 연동)
- 연락처 폼 — 메시지 입력 후 SQLite DB 에 저장
- 관리자 페이지 — 수신된 메시지 목록 조회
- 에러 페이지 — 404, 500 오류 시 커스텀 페이지 표시

1-2 전체 서버 구성



```

[ 브라우저 ]
  | HTTP Request / Response
  v
[ WEB SERVER ] Flask (Python 3.13) localhost:5000
  |           |           |
  routes.py   templates/   static/
  (라우트)   (HTML 페이지) (CSS, JS)
  |
  database.py -----> [ portfolio.db ] (SQLite3)
  |
  /api/weather -----> [ wttr.in ] (외부 API)

```

1-3 파이썬 파일 역할 분리

app.py 하나에 모든 코드를 넣는 대신, 역할별로 4 개 파일로 분리합니다.

```

app.py      Flask 앱 생성 + 서버 실행 (진입점)
config.py   설정값 + 포트폴리오 데이터
database.py DB 연결 함수
routes.py   모든 URL 라우트 + 에러 핸들러

```

이렇게 분리하면 각 파일의 역할이 명확해지고, 코드 수정이 쉬워집니다.

2 장 개발 환경 설치

2-1 필수 설치 목록

- Python 3.13 — python.org 에서 다운로드 (Add to PATH 체크 필수)
- VS Code — code.visualstudio.com 에서 다운로드
- VS Code 확장: Python, SQLite Viewer

2-2 라이브러리 설치

VS Code 터미널을 열고 아래 명령어를 실행합니다.


```
py -3.13 -m pip install flask requests
```

flask: 웹 서버 프레임워크 / requests: 외부 API 호출 라이브러리

2-3 설치 확인

```
py -3.13 -m flask --version  
# Flask 3.x.x 출력되면 정상
```

3 장 프로젝트 폴더 구조

```
Py_Project/  
|  
|-- app.py          Flask 앱 진입점  
|-- config.py       설정값 & 포트폴리오 데이터  
|-- database.py     SQLite DB 연결 함수  
|-- routes.py       URL 라우트 & 에러 핸들러  
|-- portfolio.db    SQLite 데이터베이스 파일 (자동 생성)  
|  
|-- templates/      HTML 템플릿 폴더 (Flask 규칙, 이름 변경 불가)  
|   |-- base.html   공통 레이아웃 (부모 템플릿)  
|   |-- index.html  메인 페이지  
|   |-- admin.html  관리자 메시지 페이지  
|   |-- 404.html    404 오류 페이지  
|   |-- 500.html    500 오류 페이지  
|  
|-- static/         정적 파일 폴더 (Flask 규칙, 이름 변경 불가)  
|   |-- css/  
|   |   |-- style.css  전체 스타일 (다크 네온 테마)  
|   |-- js/  
|   |   |-- main.js   날씨 위젯 JavaScript  
|   |-- images/  
|       |-- py_img.png  프로필 이미지
```

templates/ 와 static/ 폴더명은 Flask 내부 규칙이므로 반드시 이 이름을 사용해야 합니다.

4 장 설정 파일 — config.py

모든 설정값과 포트폴리오 데이터를 한 곳에 모아 관리합니다.

다른 파일들은 이 파일에서 import 해서 사용합니다.

소스코드

```
import os

# 세션 암호화 키 — 운영 환경에서는 반드시 환경변수로 교체
SECRET_KEY = os.environ.get("SECRET_KEY", "dev-secret-key-change-in-production")

# 데이터베이스 파일 경로 — config.py 와 같은 폴더에 생성
DB_PATH = os.path.join(os.path.dirname(__file__), "portfolio.db")

# 프로젝트 목록 (리스트 안에 딕셔너리)
PROJECTS = [
    {
        "title": "AI Chatbot",
        "description": "OpenAI API 기반 챗봇 시스템 구축",
        "tech": ["Python", "Flask", "OpenAI API"],
        "github": "#",
    },
    {
        "title": "Vision AI",
        "description": "이미지 인식 기반 분석 시스템",
        "tech": ["Python", "TensorFlow", "OpenCV"],
        "github": "#",
    },
    {
        "title": "Data Dashboard",
        "description": "Python 데이터 자동 분석 플랫폼",
        "tech": ["Python", "Pandas", "Matplotlib"],
        "github": "#",
    },
]
```

```

]

# 기술 스택 목록
SKILLS = ["Python", "Flask", "AI / ML", "TensorFlow", "OpenAI API", "Data Analysis"]

# 프로필 정보
PROFILE = {
    "name": "Your Name",
    "email": "example@gmail.com",
    "github": "github.com/example",
}

```

핵심 포인트

- `os.environ.get("KEY", "기본값")` — 환경변수가 없으면 기본값 사용
- `os.path.dirname(__file__)` — 현재 파일이 있는 폴더 경로를 가져옴
- `PROJECTS`, `SKILLS`, `PROFILE` 은 `routes.py` 에서 `import` 해서 템플릿으로 전달됨

5 장 데이터베이스 계층 — `database.py`

SQLite3 데이터베이스 연결과 테이블 생성을 담당하는 파일입니다.

소스코드

```

import sqlite3
from config import DB_PATH

def get_db():
    """DB 연결 객체를 반환하는 함수"""
    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row # 결과를 딕셔너리처럼 사용 가능
    return conn

```

```
def init_db():
    """서버 시작 시 테이블이 없으면 자동 생성"""
    with get_db() as conn:
        conn.execute("""
            CREATE TABLE IF NOT EXISTS messages (
                id      INTEGER PRIMARY KEY AUTOINCREMENT,
                name     TEXT NOT NULL,
                email    TEXT NOT NULL,
                message  TEXT NOT NULL,
                created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
            )
        """)
```

핵심 포인트

- `sqlite3.connect()` — DB 파일에 연결 (없으면 자동 생성)
- `row_factory = sqlite3.Row` — 결과값을 `row["name"]` 처럼 이름으로 접근 가능
- `with get_db() as conn:` — with 블록이 끝나면 자동으로 `commit + close`
- `CREATE TABLE IF NOT EXISTS` — 테이블이 이미 있으면 무시 (오류 없음)
- `AUTOINCREMENT` — id 를 1, 2, 3 ... 자동으로 증가

messages 테이블 구조

컬럼명	타입	설명
id	INTEGER	자동 증가 고유 번호 (PK)
name	TEXT	보낸 사람 이름
email	TEXT	보낸 사람 이메일
message	TEXT	메시지 내용
created_at	TIMESTAMP	전송 일시 (자동 저장)

6 장 라우트 등록 — routes.py

모든 URL 경로(라우트)와 에러 핸들러를 `register_routes(app)` 함수 안에 모아 관리합니다.

6-1 전체 라우트 목록

URL	메서드	설명
/	GET	메인 페이지
/api/weather	GET	날씨 JSON 데이터 반환
/contact	POST	연락처 폼 메시지 저장
/admin/messages	GET	관리자 메시지 목록
404 에러	-	페이지 없음
500 에러	-	서버 오류

6-2 소스코드

```
import os
import requests as http

from flask import render_template, request, redirect, url_for, flash, jsonify
from config import PROJECTS, SKILLS, PROFILE
from database import get_db

def register_routes(app):

    # — 메인 페이지 —————
    @app.route("/")
    def home():
        return render_template("index.html",
                               projects=PROJECTS,
                               skills=SKILLS,
                               profile=PROFILE)
```

```
# — 날씨 API —————
@app.route("/api/weather")
def weather():
    city = request.args.get("city", "Seoul")
    try:
        res = http.get(f"https://wttr.in/{city}?format=j1", timeout=5)
        data = res.json()
        current = data["current_condition"][0]
        return jsonify({
            "city": city,
            "temp_c": current["temp_C"],
            "desc": current["weatherDesc"][0]["value"],
            "humidity": current["humidity"],
            "feels_like": current["FeelsLikeC"],
        })
    except Exception:
        return jsonify({"error": "날씨 정보를 가져올 수 없습니다."}), 503
```

```
# — 연락처 폼 저장 —————
@app.route("/contact", methods=["POST"])
def contact():
    name = request.form.get("name", "").strip()
    email = request.form.get("email", "").strip()
    message = request.form.get("message", "").strip()

    if not all([name, email, message]):
        flash("모든 항목을 입력해 주세요.", "error")
    else:
        with get_db() as conn:
            conn.execute(
                "INSERT INTO messages (name, email, message) VALUES (?, ?, ?)",
                (name, email, message)
            )
        flash(f"{name}님, 메시지가 전송되었습니다. 감사합니다!", "success")
```

```

return redirect(url_for("home") + "#contact")

# — 관리자 메시지 목록 —————
@app.route("/admin/messages")
def admin_messages():
    pw = request.args.get("pw", "")
    admin_pw = os.environ.get("ADMIN_PASSWORD", "admin1234")
    if pw != admin_pw:
        return "접근 권한이 없습니다. (?pw=비밀번호)", 403
    with get_db() as conn:
        msgs = conn.execute(
            "SELECT * FROM messages ORDER BY created_at DESC"
        ).fetchall()
    return render_template("admin.html", messages=msgs)

# — 에러 핸들러 —————
@app.errorhandler(404)
def page_not_found(error):
    return render_template("404.html"), 404

@app.errorhandler(500)
def internal_error(error):
    return render_template("500.html"), 500

```

6-3 핵심 포인트

- `@app.route("/")` — 이 URL 로 요청이 오면 아래 함수를 실행
- `render_template()` — HTML 파일을 불러와 변수를 채워 반환
- `jsonify()` — 딕셔너리를 JSON 형식으로 반환
- `flash()` — 다음 페이지에 1 회성 알림 메시지 전달
- `redirect() + url_for()` — 다른 URL 로 이동
- `"?"` 플레이스홀더 — SQL 인젝션 방지를 위한 안전한 방법
- `request.args.get()` — URL 의 `?key=value` 파라미터 읽기

- request.form.get() — POST 폼 데이터 읽기

7 장 서버 진입점 — app.py

Flask 앱을 생성하고 서버를 실행하는 최소한의 역할만 담당합니다.

파일 분리 후 15 줄 이내로 매우 간결해졌습니다.

소스코드

```
import os                # Python 기본 제공 - 환경변수 읽기용
from flask import Flask  # flask 패키지 - Flask 클래스만
from config import SECRET_KEY  # 내 config.py - SECRET_KEY 변수만
from database import init_db  # 내 database.py - init_db 함수만
from routes import register_routes  # 내 routes.py - register_routes 함수만

app = Flask(__name__)    # Flask 앱 생성
app.secret_key = SECRET_KEY  # 세션/플래시 메시지 암호화 키

register_routes(app)     # routes.py 의 모든 라우트를 앱에 등록

if __name__ == "__main__":
    init_db()            # 서버 시작 시 DB 테이블 자동 생성
    debug = os.environ.get("FLASK_ENV", "development") == "development"
    app.run(debug=debug)  # 서버 실행
```

핵심 포인트

- Flask(__name__) — Flask 앱 객체 생성. __name__ 은 현재 파일명
- secret_key — flash() 메시지와 세션을 암호화할 때 사용
- register_routes(app) — routes.py 에 정의한 함수를 한 번에 등록
- if __name__ == "__main__": — 이 파일을 직접 실행할 때만 서버 시작
- debug=True — 코드 수정 시 자동 재시작, 오류 상세 출력 (개발용)

8 장 템플릿 — templates/

Jinja2 템플릿 엔진을 사용해 HTML 파일 안에 Python 변수와 반복문을 사용합니다.

8-1 템플릿 상속 구조

base.html (부모 — 공통 레이아웃)

```
|
|-- index.html (메인 페이지)
|-- admin.html (관리자 페이지)
|-- 404.html (404 오류)
|-- 500.html (500 오류)
```

base.html 을 수정하면 모든 페이지에 자동 적용됩니다.

8-2 base.html — 공통 레이아웃

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>{% block title %}AI Developer Portfolio{% endblock %}</title>
  <link rel="stylesheet" href="{{ url_for("static", filename="css/style.css") }}">
</head>
<body>

  {# 플래시 메시지 — flash() 로 보낸 알림을 여기서 표시 #}
  {% with messages = get_flashed_messages(with_categories=true) %}
    {% if messages %}
      <div class="flash-container">
        {% for category, message in messages %}
          <div class="flash flash-{{ category }}">{{ message }}</div>
        {% endfor %}
      </div>
    {% endif %}
  {% endwith %}
</body>
```

```

        </div>

        {% endif %}

    {% endwith %}

    {% block content %}{% endblock %}

</body>
</html>

```

8-3 Jinja2 문법 요약

<code>{{ 변수 }}</code>	Python 변수 값을 출력
<code>{% for x in list %}</code>	반복문 시작
<code>{% endfor %}</code>	반복문 끝
<code>{% if 조건 %}</code>	조건문
<code>{% endif %}</code>	조건문 끝
<code>{% extends "base.html" %}</code>	부모 템플릿 상속
<code>{% block name %}</code>	교체 가능한 영역 시작
<code>{% endblock %}</code>	교체 가능한 영역 끝
<code>{{ url_for("static", filename="css/style.css") }}</code>	정적 파일 경로 생성

8-4 index.html — 핵심 Jinja2 사용 예

```

{% extends "base.html" %}

{% block content %}

    {# 기술 스택 — skills 리스트를 반복 #}
    {% for skill in skills %}
        <div class="skill-card">{{ skill }}</div>
    {% endfor %}

    {# 프로젝트 — projects 리스트를 반복 #}
    {% for project in projects %}
        <div class="project-card">

```

```

    <h3>{{ project.title }}</h3>
    <p>{{ project.description }}</p>
    {% for tech in project.tech %}
        <span class="tag">{{ tech }}</span>
    {% endfor %}
</div>
{% endfor %}

{# 프로필 — profile 디렉터리 #}
<p>Email : {{ profile.email }}</p>
<p>GitHub : {{ profile.github }}</p>

{# 연락처 폼 — POST 방식으로 /contact 에 전송 #}
<form action="{{ url_for('contact') }}" method="POST">
    <input type="text" name="name" placeholder="이름" required>
    <input type="email" name="email" placeholder="이메일" required>
    <textarea name="message" rows="5" required> </textarea>
    <button type="submit">Send Message</button>
</form>

{% endblock %}

```

9 장 정적 파일 — static/

CSS, JavaScript, 이미지 등 서버가 가공하지 않고 브라우저에 직접 전달하는 파일들입니다.

9-1 style.css — 다크 네온 테마

전체 배경색 #0b0f1a (어두운 남색), 포인트 색상 #00f7ff (네온 청록)을 사용합니다.

```

body {
    background-color: #0b0f1a; /* 어두운 배경 */
    color: white;
}

```

```

/* 네비게이션 */
nav { display: flex; justify-content: space-between; padding: 20px 50px; }
.logo { color: #00f7ff; text-shadow: 0 0 10px #00f7ff; }

/* 히어로 섹션 */
.hero-text h2 { font-size: 60px; color: #00f7ff; }
.hero-image img { width: 320px; border-radius: 50%; border: 4px solid #00f7ff; }

/* 기술 카드 */
.skill-card { border: 1px solid #00f7ff; border-radius: 15px; }
.skill-card:hover { box-shadow: 0 0 20px #00f7ff; }

/* 프로젝트 카드 */
.project-card:hover { transform: translateY(-10px); box-shadow: 0 0 20px #00f7ff; }

/* 플래시 메시지 */
.flash-success { border: 1px solid #00f7ff; color: #00f7ff; }
.flash-error { border: 1px solid #ff4444; color: #ff4444; }

/* 날씨 위젯 */
.weather-widget { border: 1px solid #00f7ff44; border-radius: 30px; }

/* 에러 페이지 */
.error-code { font-size: 120px; color: #00f7ff; }

```

10 장 날씨 위젯 — static/js/main.js

JavaScript 로 /api/weather 엔드포인트를 호출해 실시간 날씨를 네비게이션에 표시합니다.

소스코드

```

// 날씨 설명 → 이모지 매핑 테이블
const WEATHER_ICONS = {
  "Sunny":      "☀",

```

```

"Clear":      "달",
"Partly cloudy": "구름조금",
"Cloudy":      "흐림",
"Light rain":  "비",
"Heavy rain":  "폭우",
"Light snow":  "눈",
"Thunderstorm": "천둥",
};

// 날씨 설명에 맞는 아이콘 반환
function getIcon(desc) {
  for (const [key, icon] of Object.entries(WEATHER_ICONS)) {
    if (desc.toLowerCase().includes(key.toLowerCase())) return icon;
  }
  return "온도계"; // 기본 아이콘
}

// Flask /api/weather 를 호출해 날씨 정보를 가져와 화면에 표시
async function fetchWeather(city = "Seoul") {
  try {
    const res = await fetch(`/api/weather?city=${city}`);
    const data = await res.json();

    if (data.error) throw new Error(data.error);

    document.getElementById("weather-icon").textContent = getIcon(data.desc);
    document.getElementById("weather-temp").textContent = `${data.temp_c}C`;
    document.getElementById("weather-desc").textContent = data.desc;
    document.getElementById("weather-widget").title =
      `체감 ${data.feels_like}C 습도 ${data.humidity}%`;
  } catch {
    document.getElementById("weather-desc").textContent = "날씨 정보 없음";
  }
}

```

```
// 페이지 로딩이 완료되면 자동으로 서울 날씨 조회
document.addEventListener("DOMContentLoaded", () => fetchWeather("Seoul"));
```

날씨 데이터 흐름

1. 브라우저 페이지 로드 완료
2. DOMContentLoaded → fetchWeather("Seoul") 호출
3. fetch("/api/weather?city=Seoul") → Flask /api/weather 라우트
4. Flask → wttr.in API 호출 (requests 라이브러리)
5. wttr.in → JSON 응답 반환
6. Flask → 필요한 데이터만 골라 jsonify() 로 반환
7. JS → JSON 파싱 후 HTML 요소 업데이트

11 장 관리자 페이지 — admin.html

연락처 폼으로 수신된 메시지를 테이블로 조회하는 페이지입니다.

접속 방법

```
http://localhost:5000/admin/messages?pw=admin1234
```

pw 파라미터가 없거나 틀리면 403 Forbidden 반환

운영 서버에서는 반드시 ADMIN_PASSWORD 환경변수를 설정하세요.

admin.html 소스코드

```
{% extends "base.html" %}
{% block content %}

<section class="admin-section">
  <h2>수신 메시지 <span>{{ messages | length }}건</span> </h2>
```

```

{% if messages %}
    <table class="admin-table">
        <thead>
            <tr>
                <th>#</th><th>이름</th><th>이메일</th>
                <th>메시지</th><th>수신 일시</th>
            </tr>
        </thead>
        <tbody>
            {% for msg in messages %}
                <tr>
                    <td>{{ msg.id }}</td>
                    <td>{{ msg.name }}</td>
                    <td>{{ msg.email }}</td>
                    <td>{{ msg.message }}</td>
                    <td>{{ msg.created_at }}</td>
                </tr>
            {% endfor %}
        </tbody>
    </table>
{% else %}
    <p>수신된 메시지가 없습니다.</p>
{% endif %}
</section>

{% endblock %}

```

12 장 에러 페이지 — 404 / 500

존재하지 않는 URL 접근(404)이나 서버 오류(500) 시 커스텀 페이지를 표시합니다.

routes.py 에러 핸들러

```
@app.errorhandler(404)
```

```
def page_not_found(error):  
    return render_template("404.html"), 404  
  
@app.errorhandler(500)  
def internal_error(error):  
    return render_template("500.html"), 500
```

404.html 소스코드

```
{% extends "base.html" %}  
{% block content %}  
<section class="error-page">  
    <h1 class="error-code">404</h1>  
    <p class="error-message">페이지를 찾을 수 없습니다.</p>  
    <a href="{{ url_for('home') }}"><button>홈으로 돌아가기</button> </a>  
</section>  
{% endblock %}
```

13 장 서버 실행 및 테스트

13-1 서버 실행

```
# VS Code 터미널에서 실행  
py -3.13 app.py  
  
# 정상 실행 시 출력  
* Running on http://127.0.0.1:5000  
* Debug mode: on
```


13-2 테스트 항목

URL	확인 내용
http://localhost:5000/	메인 페이지 로드
http://localhost:5000/	날씨 위젯 표시
http://localhost:5000/#contact	폼 작성 후 전송
http://localhost:5000/admin/messages?pw=admin1234	수신된 메시지 목록
http://localhost:5000/api/weather	JSON 날씨 데이터
http://localhost:5000/없는페이지	404 에러 페이지

13-3 서버 종료

터미널에서 `Ctrl + C` 를 누르면 서버가 종료됩니다.

14 장 환경 변수 및 보안

운영 환경에서는 민감한 값을 코드에 직접 쓰지 않고 환경변수로 관리합니다.

환경 변수 목록

변수명	기본값	설명
SECRET_KEY	dev-secret-key-...	세션 암호화 키
ADMIN_PASSWORD	admin1234	관리자 페이지 비밀번호
FLASK_ENV	development	운영=production

Windows 환경변수 설정 방법

```
# PowerShell (현재 세션에만 적용)
$env:SECRET_KEY = "my-super-secret-key"
$env:ADMIN_PASSWORD = "my-admin-password"

# 그 후 서버 실행
py -3.13 app.py
```

운영 환경 주의사항

- SECRET_KEY 는 반드시 긴 랜덤 문자열로 교체
- ADMIN_PASSWORD 는 단순한 admin1234 사용 금지
- portfolio.db 파일은 .gitignore 에 추가 (개인 메시지 포함)
- debug=True 는 개발용, 운영 시 FLASK_ENV=production 설정

부록 SQLite 직접 조회 방법

방법 1 — DB Browser for SQLite (GUI)

- DB Browser for SQLite 설치 후 portfolio.db 파일 열기
- Browse Data 탭에서 messages 테이블 조회
- Execute SQL 탭에서 직접 쿼리 실행 가능

방법 2 — VS Code SQLite Viewer

- VS Code 확장에서 SQLite Viewer 설치
- portfolio.db 파일을 클릭하면 테이블 내용 바로 확인

자주 쓰는 SQL 쿼리

```
-- 전체 메시지 조회 (최신순)
SELECT * FROM messages ORDER BY created_at DESC;
```

```
-- 특정 이름으로 검색
SELECT * FROM messages WHERE name LIKE "%홍길동%";
```

```
-- 메시지 개수 확인
SELECT COUNT(*) FROM messages;
```

```
-- 특정 메시지 삭제
DELETE FROM messages WHERE id = 1;
```

```
-- 전체 삭제
DELETE FROM messages;
```

```
-- 테이블 구조 확인
PRAGMA table_info(messages);
```